

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**WEB APPLICATIONS DEVELOPMENT
IT093IU
LAB 9**

Member List:

- Huỳnh Ngọc Bảo Long – ITITWE23023

Instructor: MSc. Nguyễn Trung Nghĩa

EXERCISE 6 – PASSWORD MANAGEMENT

6.1 CHANGE PASSWORD

Flow explanation

Client → PUT /api/auth/change-password

{ currentPassword, newPassword, confirmPassword }

Server → Extract current username from SecurityContext

Server → Load user from database using username

Server → Compare currentPassword with stored hashed password (BCrypt)

Server → Check newPassword == confirmPassword

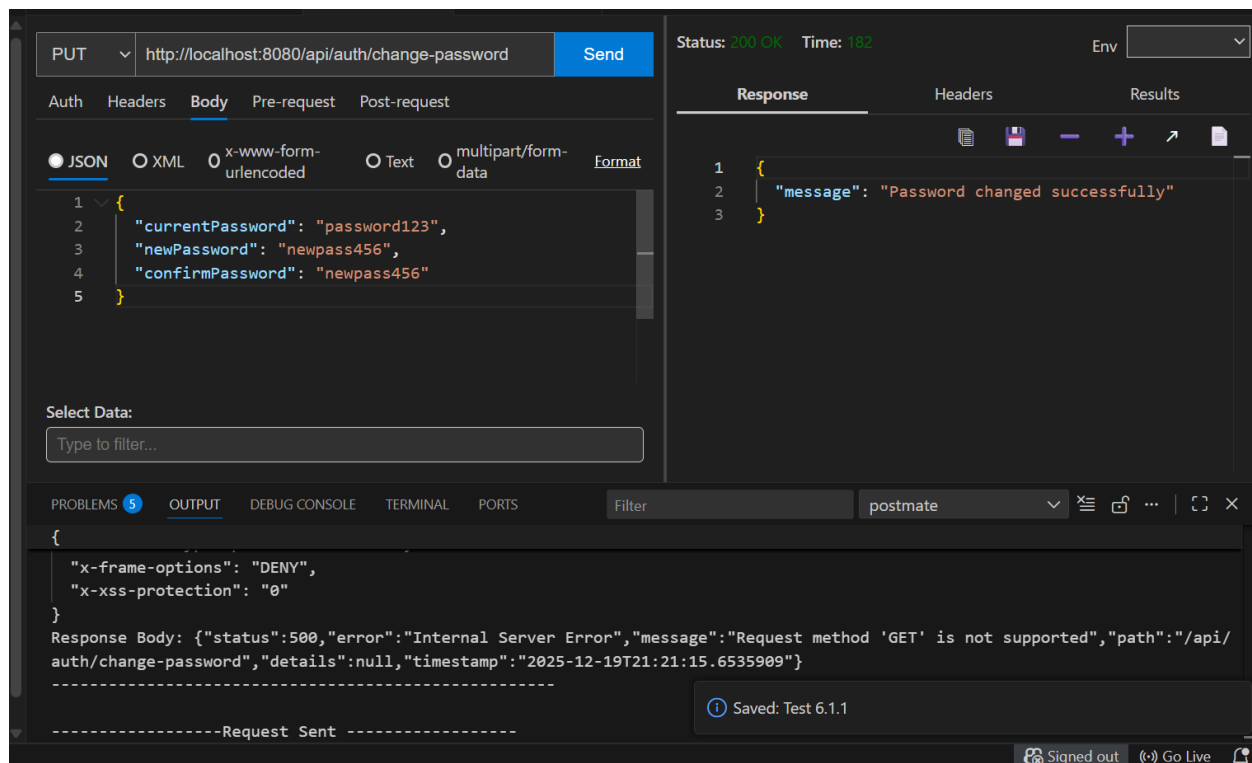
Server → Hash newPassword using BCrypt

Server → Update password in database

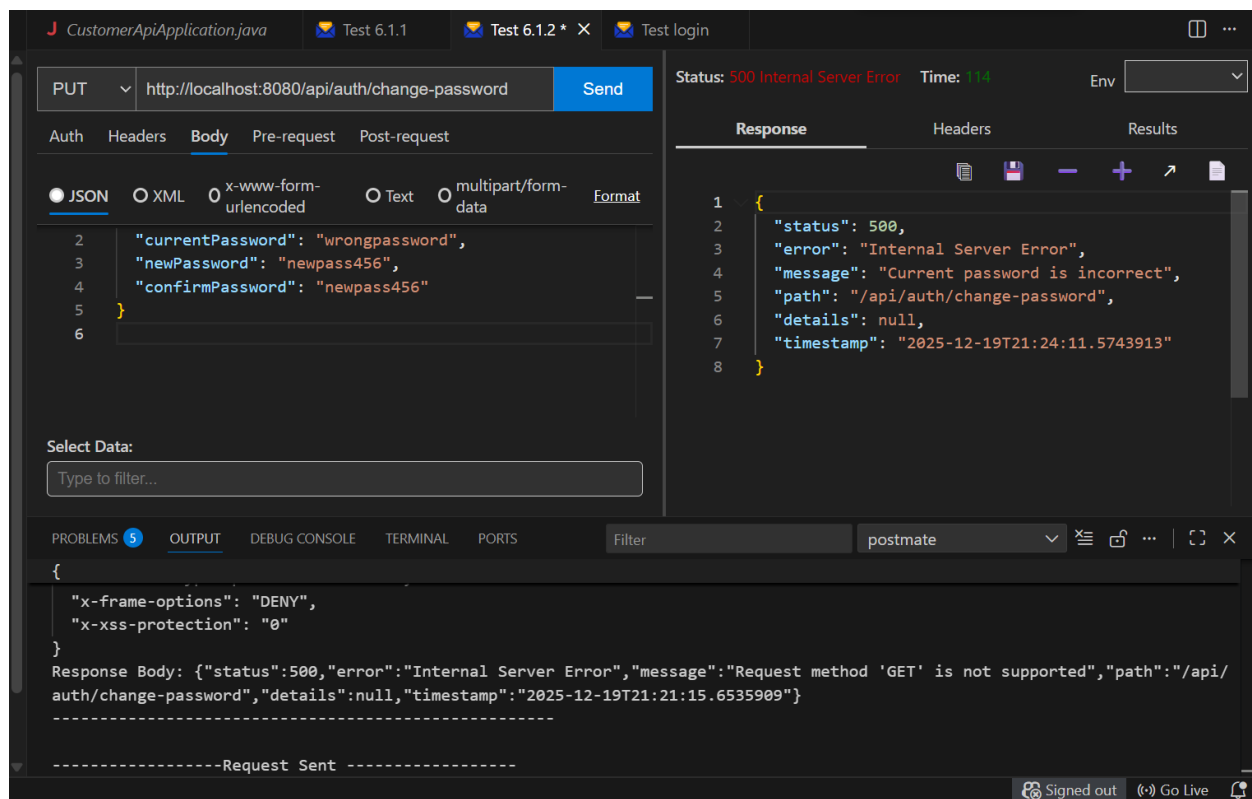
Server → Return: 200 OK { "message": "Password changed successfully" }

Test cases:

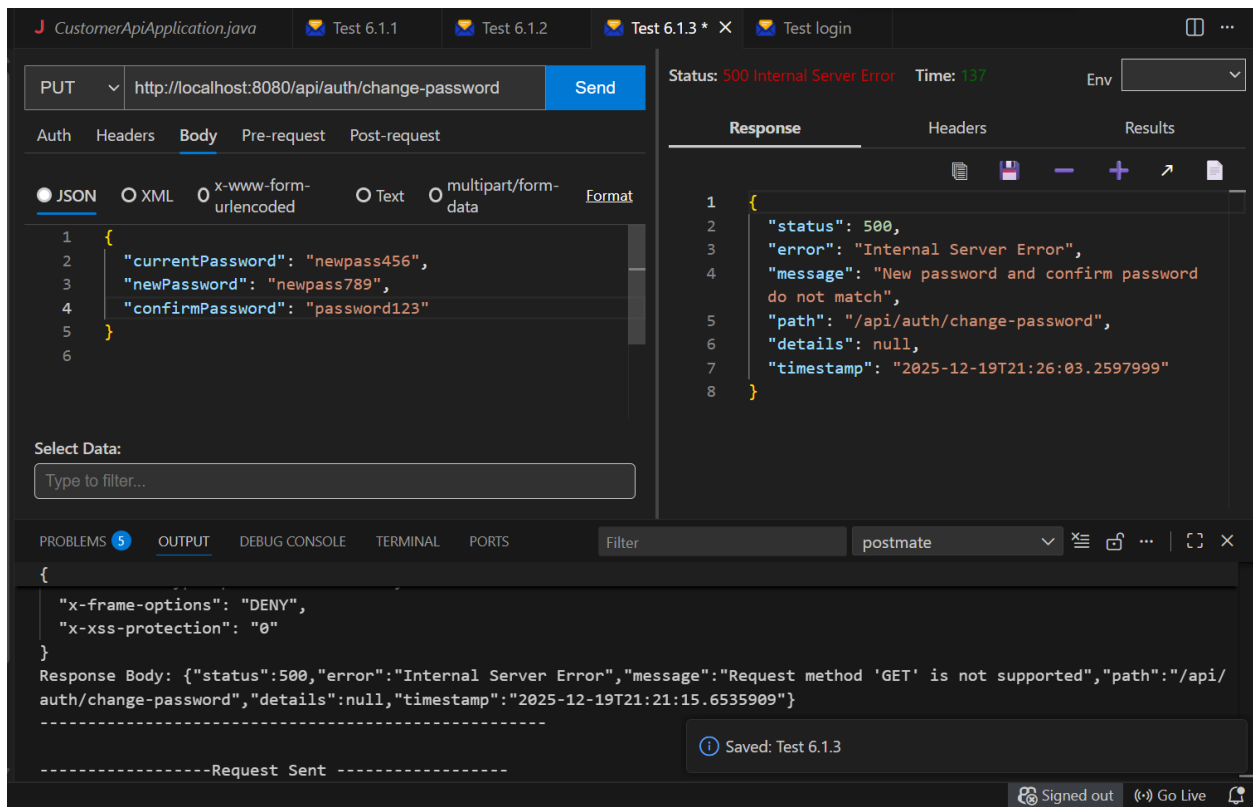
Success:



Failed:



Password confirm mismatch:



6.2 FORGOT PASSWORD

6.2.1 Generate Reset Token

Client → POST /api/auth/forgot-password

{ email }

Server → Find user by email

Server → Generate random reset token (UUID)

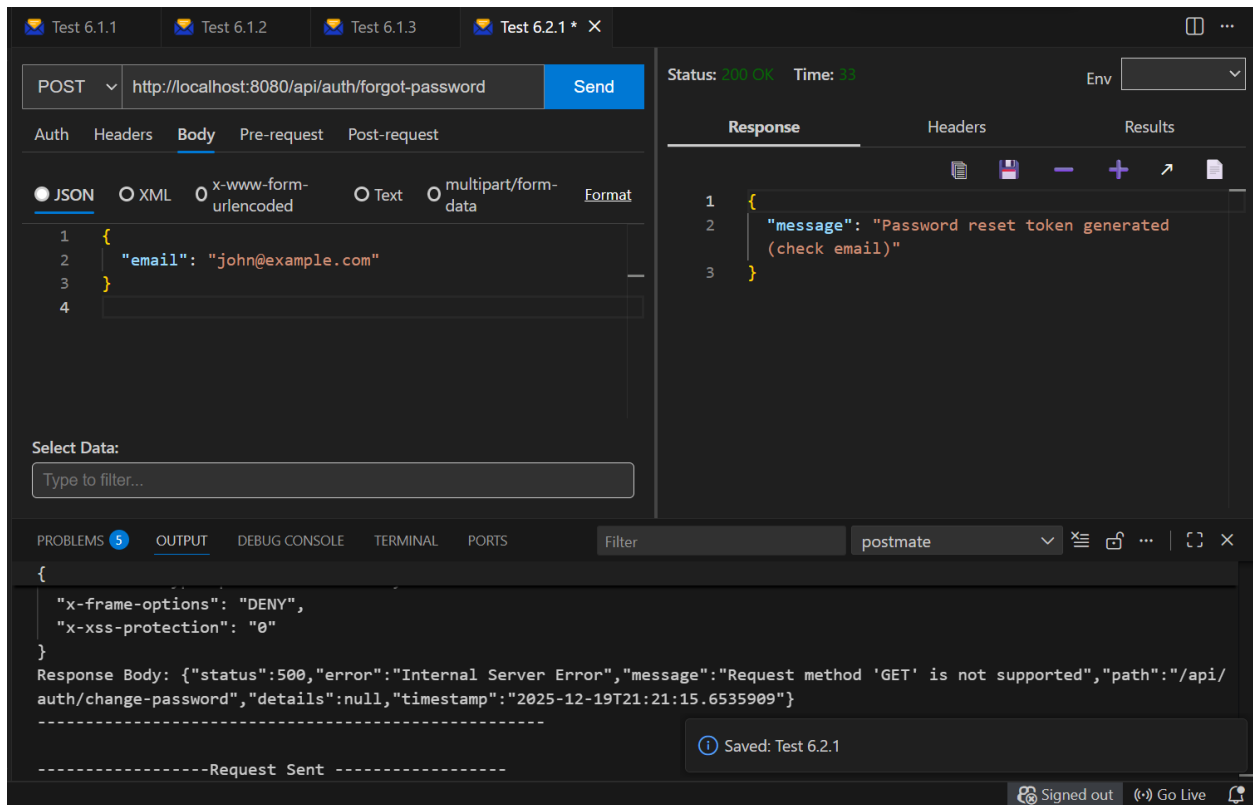
Server → Set resetToken and resetTokenExpiry = now + 1 hour

Server → Save user to database

Server → Return: 200 OK

(In real systems, token is emailed to user)

Test:



6.2.2 Reset Password

Client → POST /api/auth/reset-password

{ token, newPassword }

Server → Find user by reset token

Server → Verify token is not expired

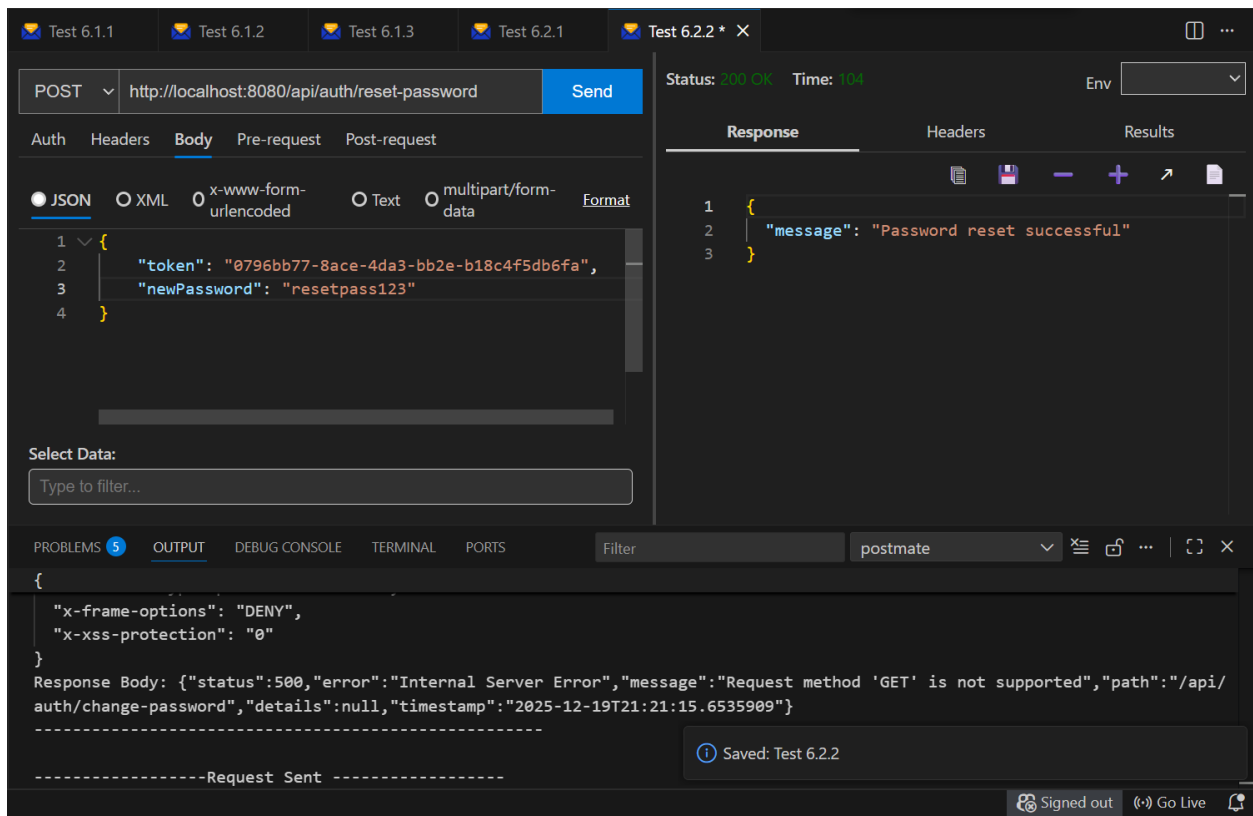
Server → Hash new password using BCrypt

Server → Update password in database

Server → Clear reset token and expiry

Server → Return: 200 OK { "message": "Password reset successful" }

Test:



EXERCISE 7 – USER PROFILE MANAGEMENT

7.1 VIEW PROFILE

Client → GET /api/users/profile

Header: Authorization: Bearer <token>

Server → JWT filter validates token

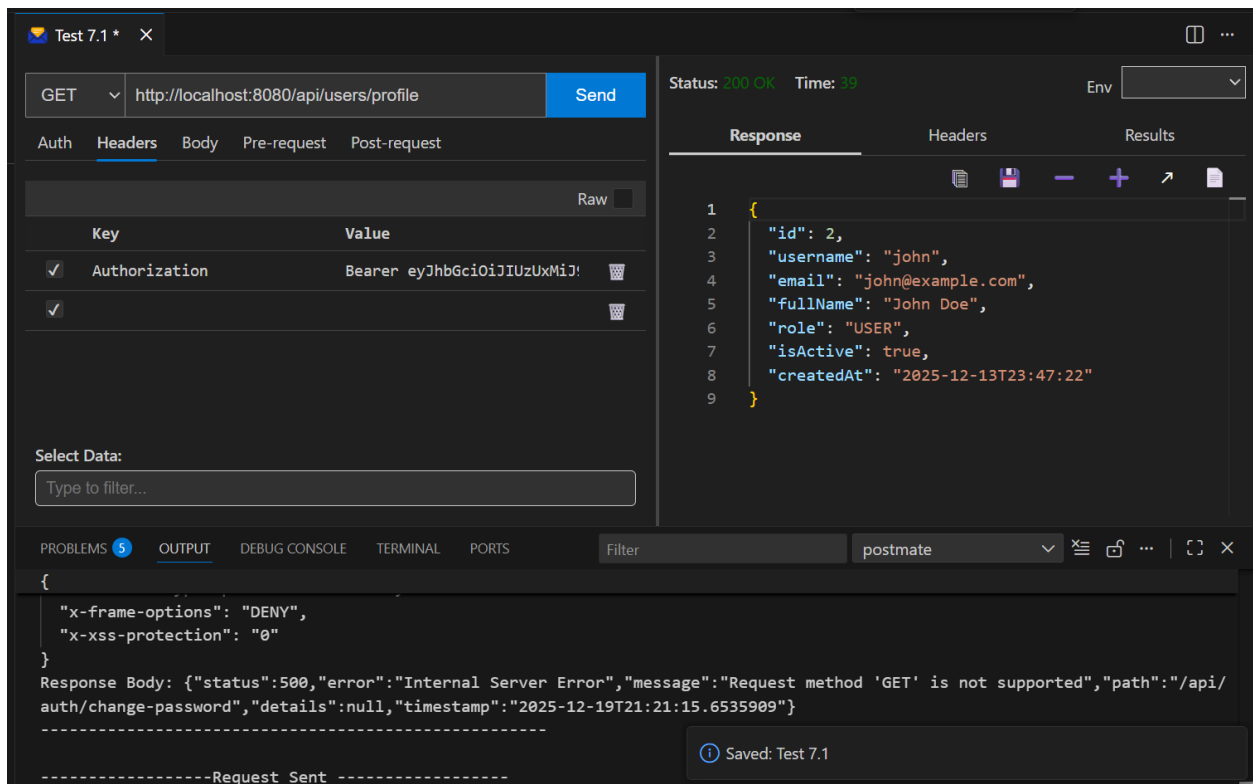
Server → Extract username from token

Server → Load user from database

Server → Convert entity to UserResponseDTO

Server → Return user profile data

Test:



7.2 UPDATE PROFILE

Client → PUT `/api/users/profile`

{ `fullName`, `email` }

Server → Extract username from `SecurityContext`

Server → Load user from database

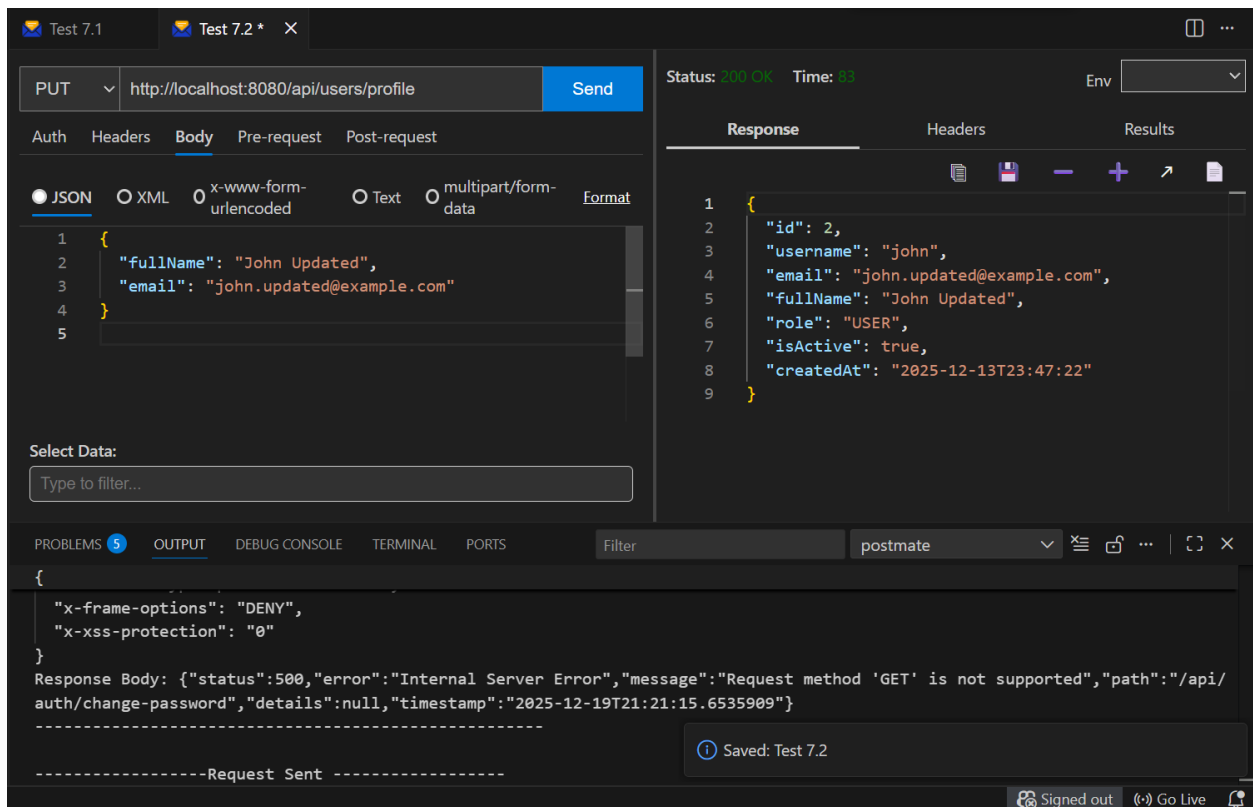
Server → Check if new email already exists

Server → Update full name and email

Server → Save updated user

Server → Return updated profile

Test:



7.3 DELETE ACCOUNT (SOFT DELETE)

Client → DELETE /api/users/account?password=...

Server → Extract username from token

Server → Load user from database

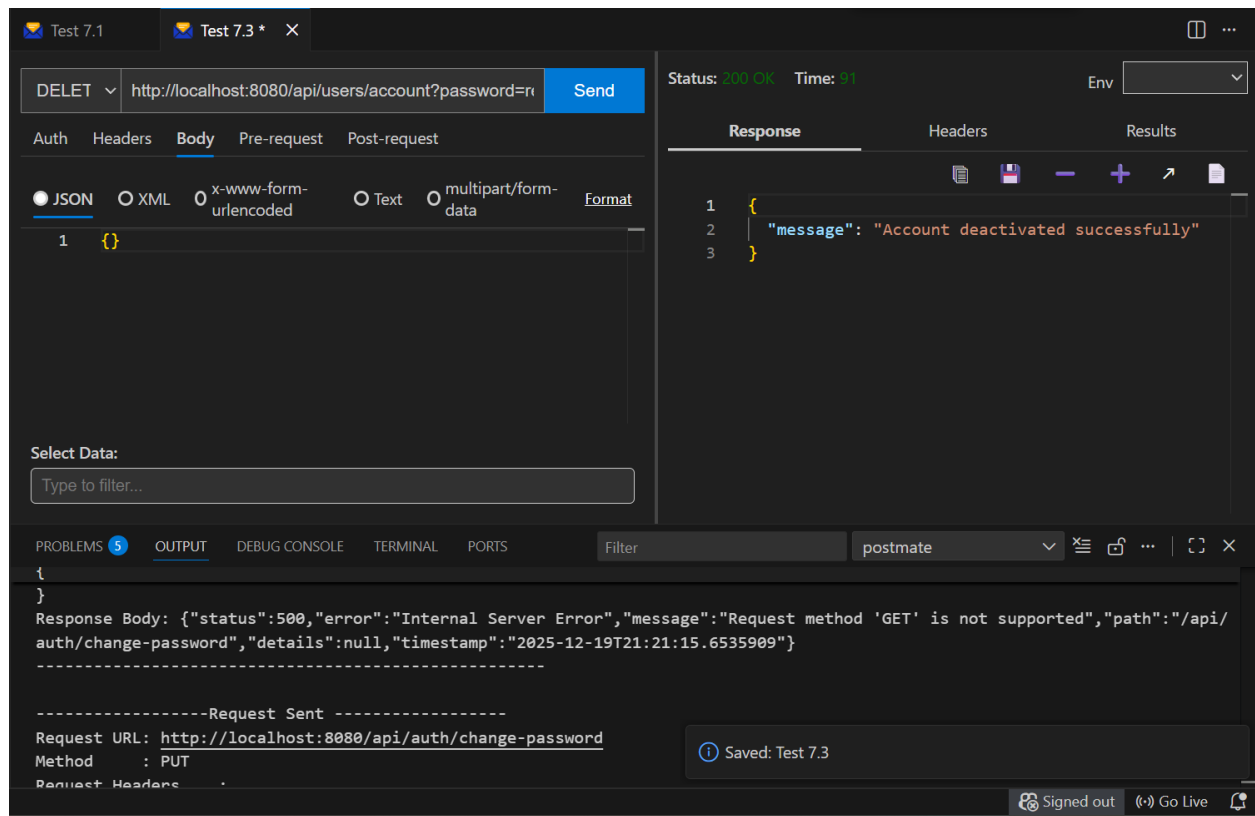
Server → Verify password using BCrypt

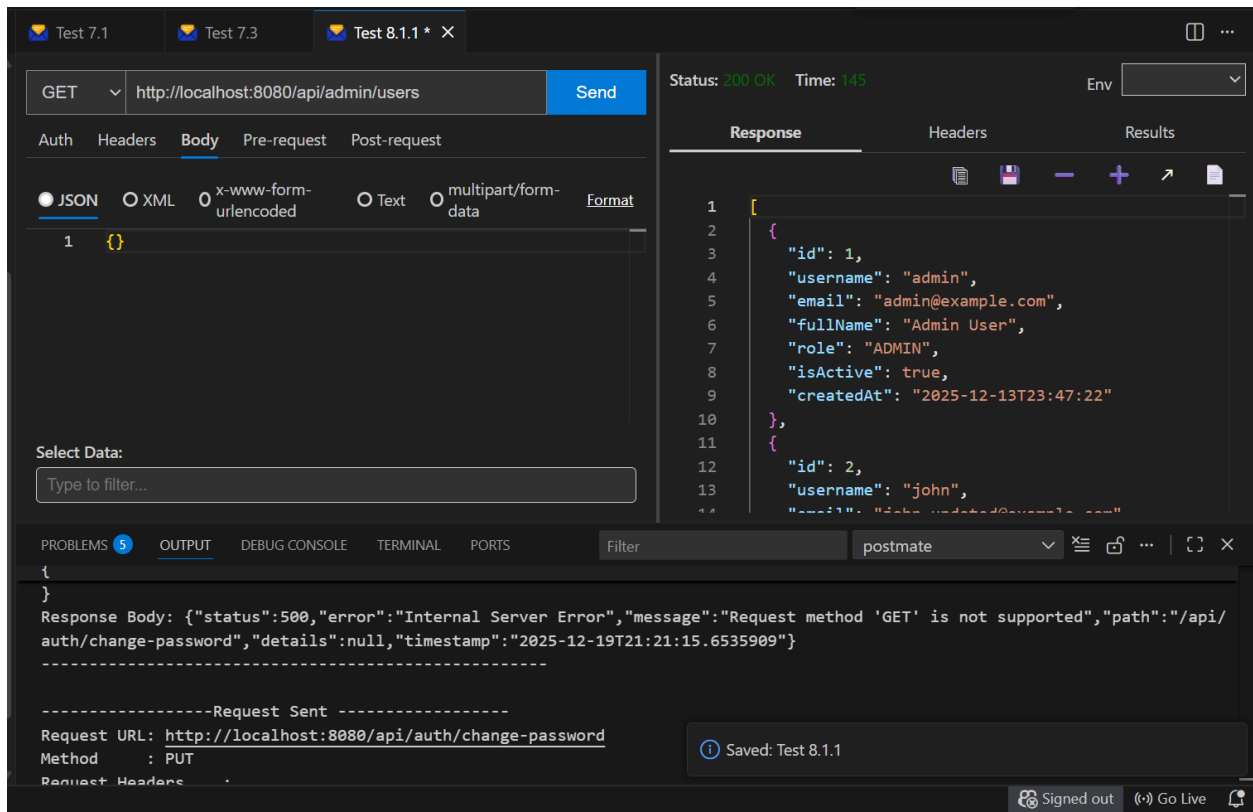
Server → Set isActive = false (soft delete)

Server → Save user

Server → Return: 200 OK { "message": "Account deactivated successfully" }

Test:





8.2 UPDATE USER ROLE

Client (ADMIN) → PUT `/api/admin/users/{id}/role`

{ role }

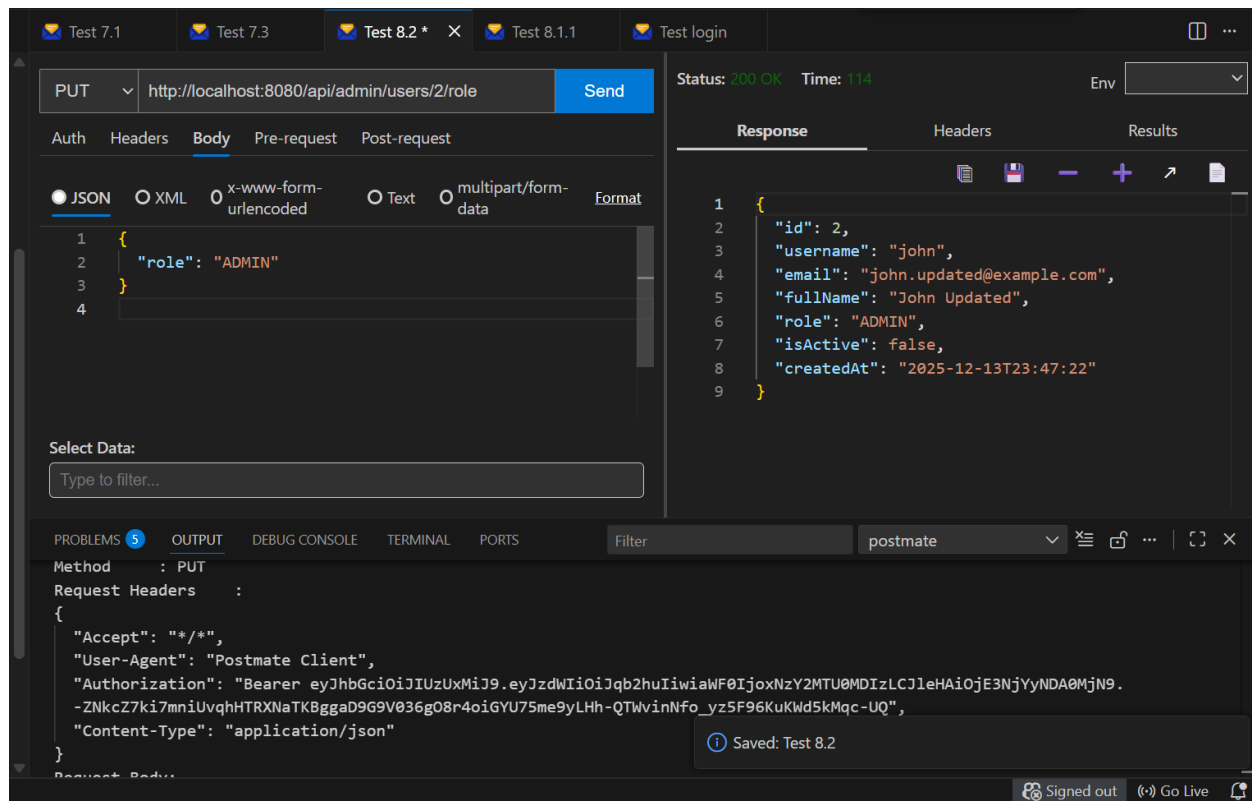
Server → Check ADMIN authorization

Server → Find user by ID

Server → Update role (USER / ADMIN)

Server → Save user

Server → Return updated user data



8.3 ACTIVATE / DEACTIVATE USER

Client (ADMIN) → PATCH /api/admin/users/{id}/status

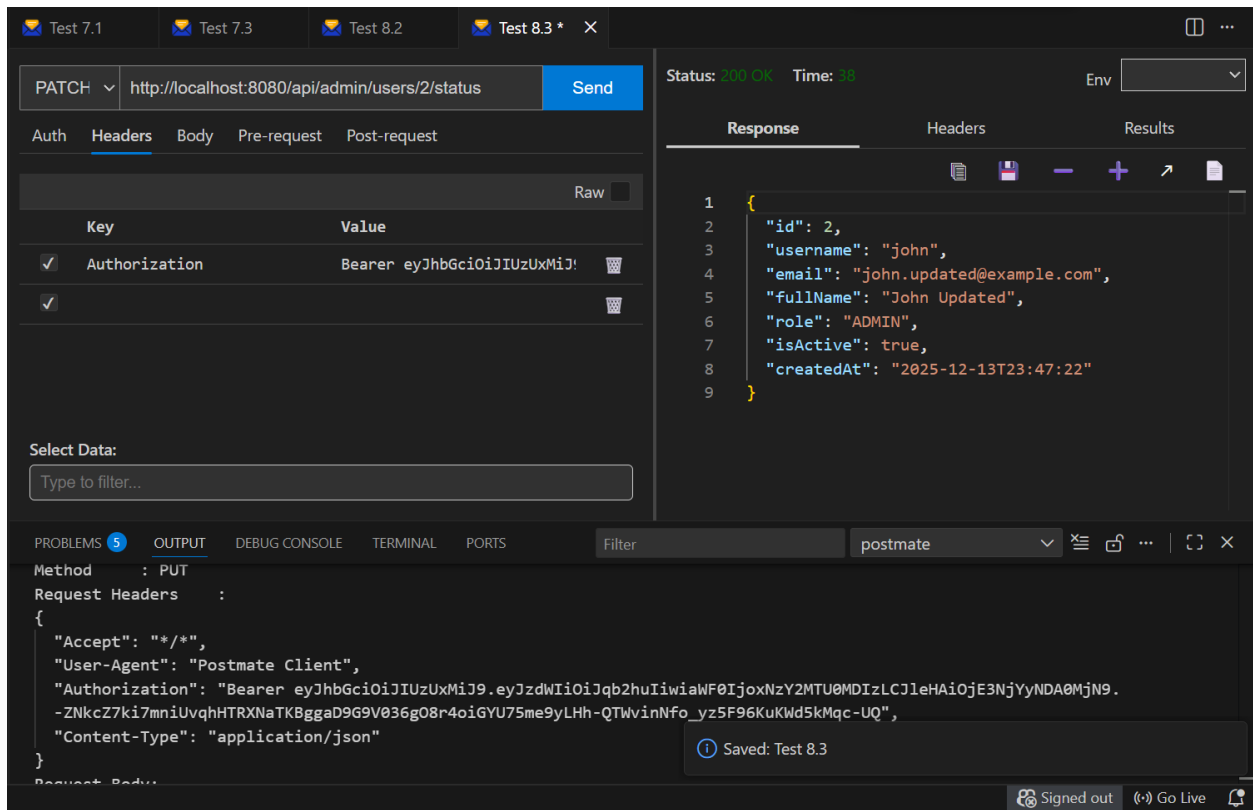
Server → Verify ADMIN role

Server → Load user by ID

Server → Toggle isActive value

Server → Save updated user

Server → Return updated user status



EXERCISE 9 – REFRESH TOKEN

9.1 LOGIN WITH REFRESH TOKEN

Client → POST /api/auth/login

{ username, password }

Server → Authenticate credentials

Server → Generate access token (short expiry)

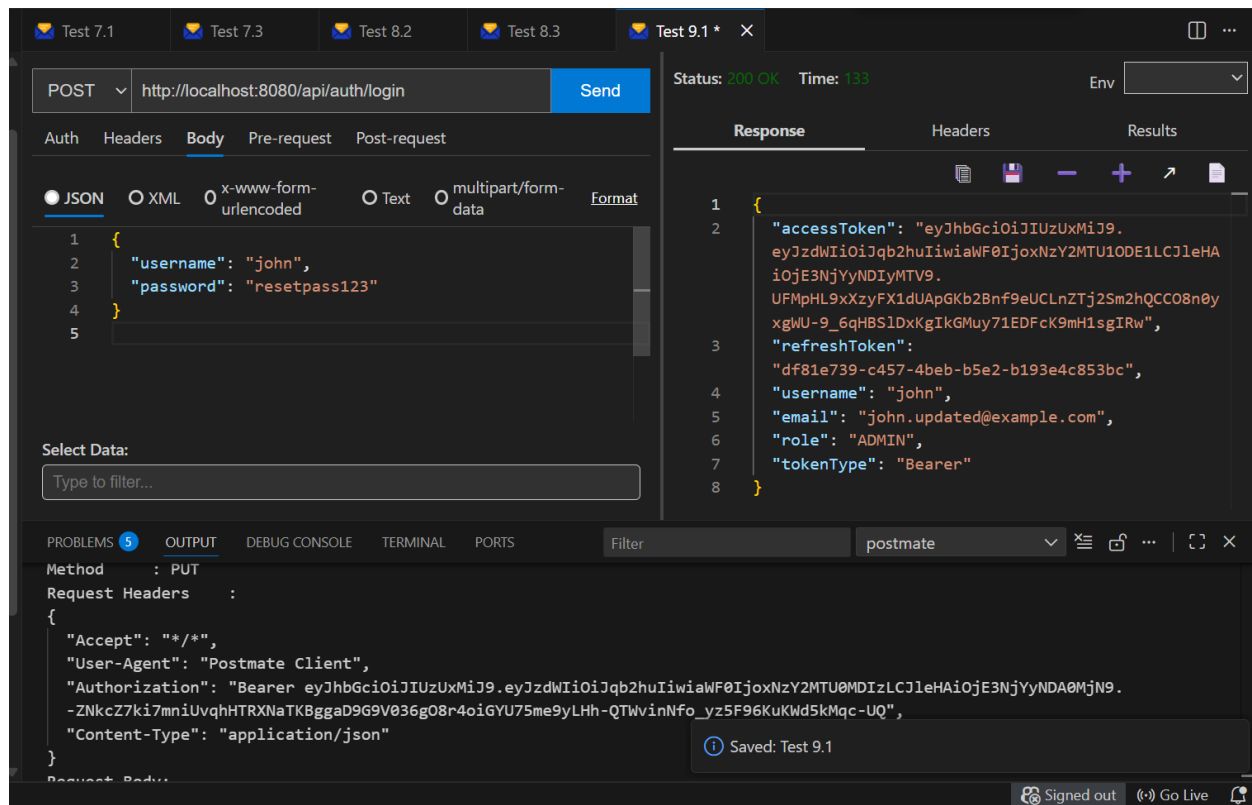
Server → Generate refresh token (long expiry)

Server → Save refresh token to database

Server → Return:

{ accessToken, refreshToken, tokenType }

Test:



9.2 REFRESH ACCESS TOKEN

Client → POST /api/auth/refresh

{ refreshToken }

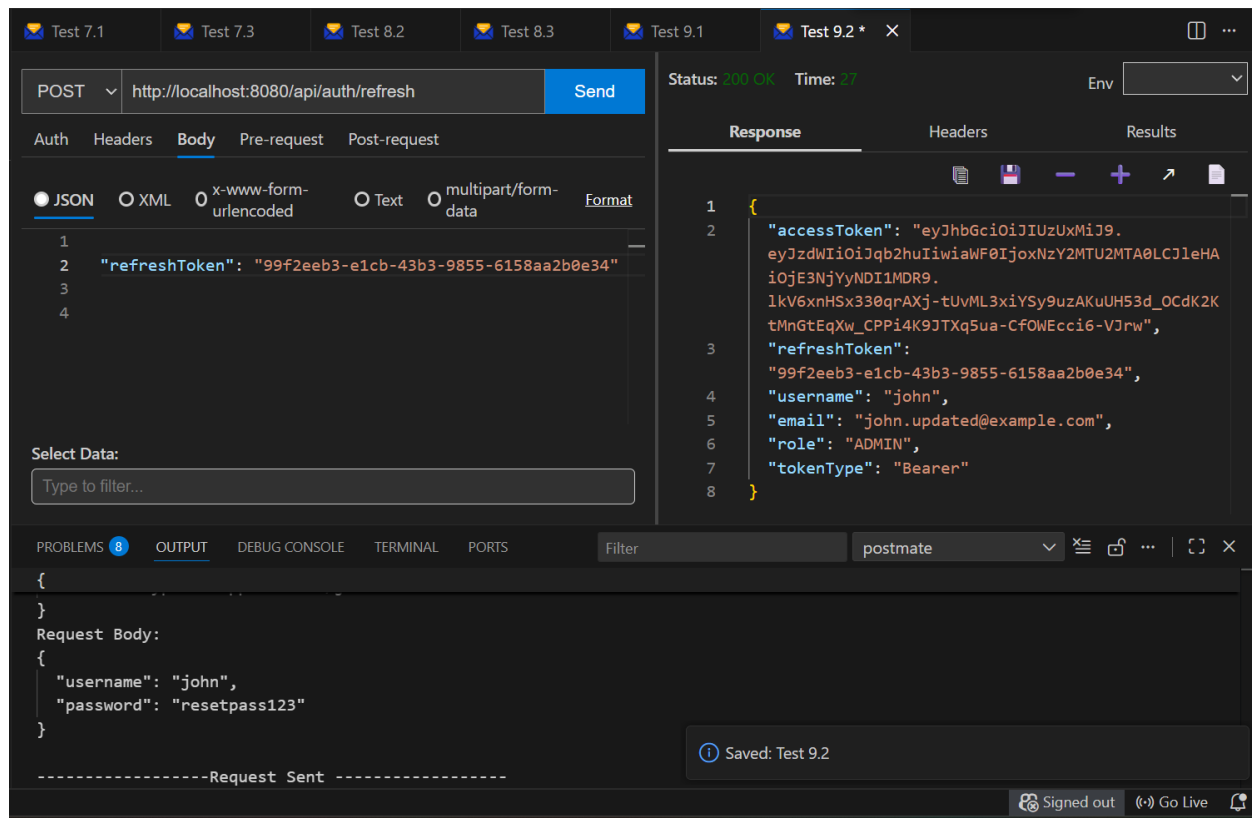
Server → Find refresh token in database

Server → Check refresh token expiry

Server → Extract user from refresh token

Server → Generate new access token

Server → Return new access token



9.3 INVALID OR EXPIRED REFRESH TOKEN

Client → Send invalid / expired refresh token

Server → Token not found or expired

Server → Reject request

Server → Return error response

Test 7.1Test 7.3Test 8.2Test 8.3Test 9.1Test 9.2Test 9.3 * X

POSThttp://localhost:8080/api/auth/refreshSend

AuthHeadersBodyPre-requestPost-request

JSONXMLx-www-form-urlencodedTextmultipart/form-dataFormat

```
1 {
2   "refreshToken": "99f2eeb3-e1cb-43b3-9855-6158aa2b0e34"
3 }
```

Select Data:

Type to filter...

Status: 404 Not FoundTime: 11Env

ResponseHeadersResults

```
1 {
2   "status": 404,
3   "error": "Not Found",
4   "message": "Invalid refresh token",
5   "path": "/api/auth/refresh",
6   "details": null,
7   "timestamp": "2025-12-19T21:57:29.5771744"
8 }
```

PROBLEMS8OUTPUTDEBUG CONSOLETERMINALPORTSFilterpostmate

```
{
}
Request Body:
{
  "username": "john",
  "password": "resetpass123"
}
```

Saved: Test 9.3

Saved: Test 9.1

-----Request Sent -----

Signed outGo Live